
Chapter 5 | 整机集成与端到端推理

- 本章目标：Hack CPU + NPU 组成完整系统。
- 内容：MMIO 扩展、Hack 驱动、集成测试。
- 最终成果：一条命令完成 MNIST 分类。

Chapter 5 | Hack + NPU 系统

最后一步是把 NPU 挂进 `n2t\_computer`：Memory 在地址译码时增加 0x7000-0x7FFF 段，命中该段就把读写转给 NPU。CPU 跑一条普通汇编指令就能触发 NPU。

对 CPU 而言 NPU 只是“一块特殊的内存”；对 NPU 而言 CPU 只是“一个会读写寄存器的控制器”。这就是 MMIO 的对称性。

把 NPU 挂进 Hack 就像给自行车加了一个电机：骑车的人（CPU）还是控制方向，但爬坡（矩阵计算）交给电机（NPU）。

改造点只有一个：Memory 在地址译码时多认一段地址，这段地址的读写全部转给 NPU。对 CPU 来说，“让 NPU 干活”和“写内存”是同一件事。

- CPU：运行 Hack 机器码，负责调度。
- Memory：RAM/Screen/Keyboard + NPU MMIO。
- NPU：独立时钟域或同源时钟，完成计算。
- 数据流：ROM 程序 -> CPU -> MMIO -> NPU -> RAM。

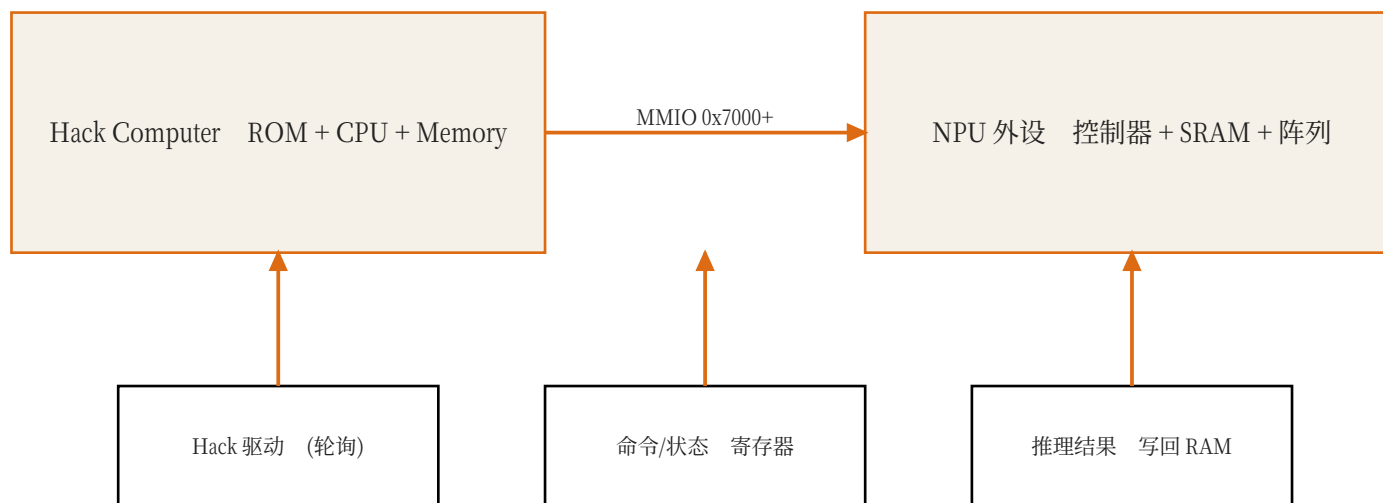


图 14：NPU 作为内存映射外设挂在 Hack 整机上。

Chapter 5 | MMIO 地址分配

Hack 原始地址空间：0x0000-0x3FFF 是 RAM，0x4000-0x5FFF 是 Screen，0x6000 是 Keyboard。0x7000 以上未使用，所以 NPU 占用这一段不会破坏原有程序。

寄存器表建议：0x7000 CMD、0x7001 STATUS、0x7002 ADDR_SRC、0x7003 ADDR_DST、0x7004 LENGTH。具体位宽在 Ch1 Assignment 里设计。

地址就像小区门牌：0x0000-0x3FFF 是居民楼（RAM），0x4000-0x5FFF 是商场屏幕（Screen），0x6000 是门卫室（Keyboard），0x7000 以上是新建的 NPU 厂房。

只要新地址不占老门牌，原来的居民（旧程序）就完全不受影响。这也是为什么把 NPU 放在 0x7000+。

- 0x0000-0x3FFF：RAM16K。
- 0x4000-0x5FFF：Screen。
- 0x6000：Keyboard。
- 0x7000-0x7FFF：NPU 寄存器与缓冲区（新增）。

图 2：Hack 地址空间。0x7000 以上原本未使用，
正好预留给 NPU。



Chapter 5 | 命令协议

握手协议要简单可靠：CPU 先写参数，再写 $CMD=START$ ；NPU 收到后把 $BUSY$ 置 1，开始执行；完成后 $BUSY$ 清零、 $DONE$ 置 1；CPU 轮询到 $DONE$ 后读结果，最后写 $CMD=CLR$ 清状态。

这个协议不需要中断，也不需要新的 Hack 指令，完全用现有汇编读写内存实现。

协议就是“先做什么、再做什么”的约定。我们的约定是：先填地址参数，再按“开始”按钮（写 CMD ），NPU 亮“工作中”灯（ $BUSY$ ），算完亮“完成”灯（ $DONE$ ），CPU 看到灯后取结果。

协议越简单越不容易出错。真实芯片里的握手协议通常也是这个模式的加强版。

- CPU 把图片地址、权重地址、层数写入寄存器。
- CPU 写 $CMD=START$ ，NPU 开始执行。
- NPU 执行时置 $BUSY=1$ 。
- 完成后 $BUSY=0$ 、 $DONE=1$ ，结果写入约定地址。

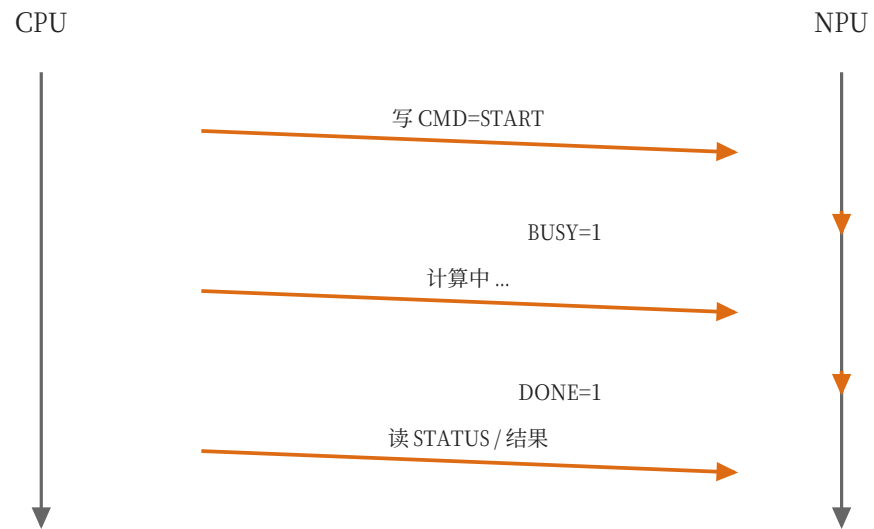


图 15：先写命令，再轮询完成位，最后读结果。

Chapter 5 | Hack 驱动

Hack 汇编驱动核心就几行：把地址常数加载到 A，用 `M=D` 写寄存器，用 `D=M` 读寄存器，用跳转指令循环轮询。

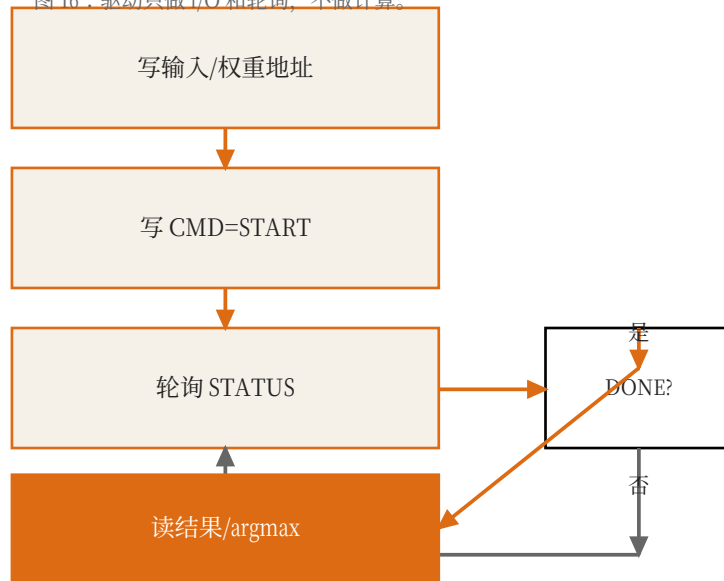
驱动里没有乘法，也没有矩阵运算——那些都在 NPU 里。驱动只负责“把任务描述清楚”和“等结果”。

驱动就是写给 CPU 的“操作手册”：把图片地址写到 0x7002，把结果地址写到 0x7003，向 0x7000 写 1，然后反复读 0x7001 直到完成位出现。

Hack 没有乘法指令，但没关系——驱动里不需要乘法。它只是“传话的人”，真正的数学在 NPU 里。

- 汇编伪代码：写 ADDR_SRC、写 ADDR_DST、写 CMD、轮询 STATUS。
- Hack 没有函数调用栈，用小标签和跳转即可。
- 驱动只做 I/O 和地址计算，不做矩阵运算。
- 完成后读取结果 RAM，比较 argmax。

图 16：驱动只做 I/O 和轮询，不做计算。



Chapter 5 | 轮询 vs 中断

轮询 (polling) 就是 CPU 反复读 STATUS，直到完成位出现。实现简单，但 CPU 在等待期间不能干别的。

中断 (interrupt) 需要硬件在完成时打断 CPU，Hack 指令集没有中断机制，所以本章用轮询。真实系统中小任务轮询也完全够用。

轮询像你在洗衣机前一直盯着“剩余时间”，直到显示 0。简单，但期间你什么别的事都干不了。

中断像洗衣机洗完会“叮”一声叫你，你可以先去看书。Hack 没有“叮”这个硬件机制，所以我们用轮询。

- 轮询：CPU 循环读 STATUS，简单可靠。
- 中断：需要扩展 Hack 指令集或外设引脚，复杂。
- 本章用轮询，符合 nand2tetris 的极简风格。
- 真实 SoC 中轮询对小任务完全够用。

Chapter 5 | 集成测试

集成测试的输入是 Ch4 导出的一张照片和权重文件；仿真启动时通过`\$readmemh`预置到 SRAM/RAM。CPU 跑驱动，NPU 算完，测试台读取最终结果并检查 argmax。

验收标准两条：原有`make test`必须全绿（不能破坏 Hack 语义），新增端到端测试必须 PASS。

集成测试就是“全流程彩排”：图片、权重、程序全部就位，CPU 按剧本（驱动）走，NPU 按剧本算，最后检查输出类别是不是标准答案（golden）。

同时还要跑一遍旧考试（make test），确保加了 NPU 之后，原来的 Hack 电脑没有被改坏。

- 用 Ch4 的一张测试图片作为输入。
- NPU 权重在仿真启动时从文件加载。
- CPU 跑驱动，NPU 跑推理。
- testbench 检查最终类别并打印 PASS。

Paper: Jouppi et al., In-Datacenter Performance Analysis of a Tensor Processing Unit, ISCA 2017

Chapter 5 | 调试方法

- 先单独测 MMIO 读写，再测 NPU 状态机。
- 用 VCD 波形检查 CMD/STATUS 握手。
- 逐层比对 NPU 输出与 golden。
- 不一致时从最后一层往前定位。

Chapter 5 | 性能度量

- 统计从 START 到 DONE 的周期数。
- 对比理论 MAC 周期数，算出阵列利用率。
- 瓶颈通常在 DMA/权重加载，而不是阵列。
- 利用率报告是本章加分项。

Chapter 5 | 扩展方向

MNIST 之后，同一个框架可以换数据集、加深网络，也可以评估 tiny YOLO：卷积和 FC 直接复用，需要新增的是 upsample、concat、检测头和后处理 NMS。

NMS 这类控制密集的后处理可以留在 CPU 或 Python，NPU 只做最重的张量运算，这是真实 SoC 的常见分工。

MNIST 是“小样”，证明整条流水线能跑通。换成 tiny YOLO 时，卷积、FC、池化都能复用，新增的是上采样、拼接、检测头和 NMS。

真实系统里 NMS 这类“判断多、计算少”的活通常留给 CPU，NPU 只做“计算多、判断少”的张量运算。

- 把 MNIST 换成 CIFAR-10 或自定义数据集。
- 增加批量推理与流水。
- 评估 tiny YOLO 的 conv 部分。
- NMS 等后处理可以继续留在 CPU/Python。

Paper: Redmon & Farhadi, YOLOv3: An Incremental Improvement, arXiv 2018

Chapter 5 | 本章任务

- 扩展 Memory/Computer 的 MMIO。
- 写 Hack 驱动并编译。
- 跑通端到端 MNIST 推理。
- 保持全部历史测试通过。

Chapter 5 | 总结

- 从 Nand 到 Hack, 再到 NPU：一条完整的现代计算机链路。
- MMIO 是 CPU 与外设之间的桥梁。
- 小分类模型是端到端验证的完美载体。
- 后续可以按同样框架挑战 tiny YOLO。

术语速查 (Glossary)

- CPU：中央处理器，负责执行指令；在本课程里是 Hack CPU。
- RAM：随机存取存储器，Hack 有 64KB，用于放数据和程序状态。
- MMIO：Memory-Mapped I/O，把外设寄存器映射到内存地址，CPU 用普通读写指令控制外设。
- 寄存器：CPU 或外设内部的小容量存储单元，通常 8/16/32 位。
- SRAM：片上静态随机存储器，速度快、容量小，NPU 用它暂存权重和特征图。
- MAC：乘累加运算： $acc = acc + a * w$ ，是 CNN 最基础的计算。
- GEMM：通用矩阵乘， $C = A \times B$ ；卷积可以转换成 GEMM。
- PE：Processing Element，处理单元；本课程里是一个 MAC 单元。
- Systolic Array：脉动阵列：PE 排成网格，数据在相邻 PE 间逐拍流动。
- Weight-Stationary：权重驻留式数据流：权重装载后停在 PE 内反复使用。
- 斜输入：把第 row 行输入延迟 row 拍，使阵列各行在同一拍处理同一个 k 。
- im2col：Image to Column，把卷积窗口展平成矩阵行，从而用 GEMM 计算卷积。
- Line Buffer：行缓冲：只保留最近若干行，供滑动窗口复用。
- FSM：有限状态机：一组状态和跳转条件，NPU 控制器用它实现调度。
- DMA：Direct Memory Access，直接内存访问，批量搬运数据。

-
- 量化：把浮点数映射到低比特整数（如 int8），并记录 scale 以便还原。
 - Scale：量化比例因子，决定一个整数单位代表多大的浮点数值。
 - Polling：轮询：CPU 反复读状态寄存器，直到外设完成。
 - argmax：取最大值的下标；分类任务里就是预测类别。
 - NPU：Neural Processing Unit，神经网络处理器，专为张量运算设计的加速器。

References / 延伸阅读

- Patterson & Hennessy, "Computer Organization and Design" (memory-mapped I/O) .
- Jouppi et al., "In-Datacenter Performance Analysis of a Tensor Processing Unit", ISCA 2017.
- Redmon et al., "You Only Look Once: Unified, Real-Time Object Detection", CVPR 2016.
- Redmon & Farhadi, "YOLOv3: An Incremental Improvement", arXiv 1804.02767, 2018.